

ByteToken: Lossless Binary Encoding for Tokenized LLM Interfaces Using Concatenation-Safe Token Alphabets

Chandan Pandey

<https://github.com/chandanpandey/bytetoken>

Draft preprint, August 2026

Abstract

Tokenized language-model interfaces are inefficient carriers for arbitrary binary payloads when those payloads are first converted to conventional textual encodings such as Base64. This paper studies a narrower alternative: construct a binary-to-token representation from tokenizer vocabulary items that are empirically validated to remain independently decodable under the required serialization boundary.

ByteToken implements fixed-width encodings for selected tokenizer configurations. The standard string mode maps fixed-width bit chunks to space-prefixed vocabulary items that are tested for tokenizer stability. A second mode operates directly on round-trip-safe token IDs and therefore avoids string re-tokenization, but it requires an inference interface that accepts pre-tokenized IDs; this should not be assumed for hosted text APIs. We also implement a conservative shared-alphabet mode over the tested `cl100k_base` and `o200k_base` encodings.

The central theoretical result is intentionally scoped: for any fixed alphabet of N independently decodable, concatenation-safe symbols, a fixed-width code carries at most $\lfloor \log_2 N \rfloor$ bits per emitted symbol, and ByteToken reaches that bound by selecting 2^b symbols. This is not a global optimality result over variable-length, stateful, multi-token, or API-specific encodings.

The implementation demonstrates lossless local round-trips and provides reproducible benchmark scripts. The current developer-payload benchmark is synthetic; conventional compression accounts for much of the total reduction on structured data, so compression and binary-to-token encoding must be reported separately. We present ByteToken as an experimental transport primitive rather than a method for making language models reason over compressed binary data.

1 Introduction

Modern language-model APIs meter and constrain requests in token units. When arbitrary bytes must cross a text-oriented boundary, Base64 is a common representation because it is portable and lossless [3]. Base64 expands the byte stream before tokenization, and a model tokenizer may then split the resulting characters into additional subword tokens.

ByteToken asks whether a tokenizer’s existing vocabulary can itself serve as a denser transport alphabet. If a set of vocabulary items can be serialized next to one another without changing their token boundaries, each item can act as a code symbol. A 2^b -symbol alphabet can represent b input bits per emitted symbol.

The practical difficulty is not bit packing. It is identifying symbols whose behavior remains stable under the exact tokenizer and serialization path being used. Tokenizers are implementation- and version-specific, and a condition sufficient for one tokenizer need not generalize to another.

The contributions of this work are deliberately limited to:

1. a concrete lossless binary-to-token encoding construction;

2. empirical alphabet discovery for selected `tiktoken` encodings;
3. a negative result showing that simple self-concatenation checks are insufficient to justify arbitrary string concatenation for all candidate tokens;
4. a direct-token-ID representation that separates token-density experiments from text serialization;
5. an elementary fixed-alphabet density bound stated with explicit assumptions; and
6. open-source code, tests, and benchmark scripts intended for reproduction and criticism.

2 Background

2.1 Subword tokenization

Byte Pair Encoding (BPE) and related subword methods represent text as sequences drawn from a learned vocabulary [10]. SentencePiece provides a language-independent tokenization framework with explicit handling of whitespace boundaries [4]. Modern model tokenizers are not interchangeable: vocabulary contents, merge behavior, normalization, special-token handling, and versioning can all affect whether an encoded string round-trips as intended.

ByteToken’s current primary implementation uses OpenAI’s open-source `tiktoken` library [6]. The paper’s empirical statements should be read as statements about the tested encoding/library configurations, not as axioms of BPE in general.

2.2 Binary-to-text encodings

RFC 4648 defines Base16, Base32, and Base64 [3]. These encodings are designed for textual transport, not for minimizing a particular language-model tokenizer’s token count.

Higher-radix Unicode encodings such as Base32768 [5] show that a Unicode code point can carry many input bits. That does not imply equivalent language-model token density: a tokenizer may split a Unicode character or merge neighboring characters in ways that change the number of model tokens.

2.3 Context compression is a different problem

Methods such as LLMingua [2] compress natural-language prompts while attempting to preserve task-relevant semantics. In-context autoencoding likewise targets semantic context compression [1]. ByteToken solves a different problem: exact byte preservation for payloads that must be transported through a tokenizer-backed representation.

Conventional compressors such as LZMA operate on the byte stream itself [8]. They are complementary to ByteToken. If a structured payload compresses strongly, most end-to-end savings may come from compression rather than from the final binary-to-token encoding.

Token-free and byte-level model architectures such as ByT5 [11] and the Byte Latent Transformer [7] further illustrate that the tokenization boundary is architectural rather than fundamental. A system that accepts raw bytes natively does not need ByteToken for byte transport.

3 Encoding Model

Let T denote a deterministic tokenizer mapping text to token IDs, and let D denote the corresponding token decoder.

3.1 Atomic and concatenation-safe symbols

For this paper, a candidate token ID t is *round-trip atomic* if

$$T(D([t])) = [t].$$

For a string-mode alphabet A , the operational requirement is stronger: concatenating the decoded forms of symbols in the supported encoding must re-tokenize to the same sequence of selected token IDs.

The implementation uses conservative filters for string mode, including a space-prefixed candidate set. This is an empirical engineering rule for the tested encodings. We do **not** claim that leading spaces create a universal BPE invariant.

3.2 Fixed-width encoding

Choose 2^b validated symbols and assign each symbol an integer in $[0, 2^b - 1]$. The encoder converts input bytes to a bit stream, stores the final padding length in a metadata symbol, splits the stream into b -bit chunks, and maps each chunk to its assigned symbol. Decoding reverses the mapping and removes padding.

3.3 Modes in the implementation

Mode	Representation	Current scope
Standard	Text	Conservative space-prefixed candidates; 15-bit default; tested primarily with <code>c1100k_base</code> and <code>o200k_base</code> .
Shared alphabet	Text	Intersection across the explicitly configured/tested tokenizer set. “Universal” is a historical class name, not an all-model guarantee.
Direct ID	Token IDs	16-bit operation for <code>c1100k_base</code> and 17-bit operation for <code>o200k_base</code> in the current implementation/scans; requires an interface that preserves token IDs directly.
SentencePiece	Text	Experimental path validated on the included test model; not a claim covering all SentencePiece models.

Table 1: ByteToken representations and their intended scope.

The `DirectIDEncoder.encode_to_string()` helper serializes token IDs as JSON for storage/debugging. That JSON representation is not the high-density Direct-ID transport representation.

4 A Scoped Density Bound

Proposition 1. Let A be a set of N independently decodable symbols available to a fixed-width encoder. If each emitted symbol independently selects one member of A , then the largest integer number of payload bits that can be represented by one emitted symbol is

$$b_{\max} = \lfloor \log_2 N \rfloor.$$

Proof. A b -bit input chunk has 2^b possible values. An injective fixed-width mapping from all such chunks to A requires $2^b \leq N$. Therefore $b \leq \log_2 N$, and the largest integer b is $\lfloor \log_2 N \rfloor$. Selecting any 2^b members of A constructs a code that reaches the bound. $\square$

Scope. This proposition does not establish global information-theoretic optimality for tokenizer-aware binary transport. It does not cover variable-length codes, stateful encoders, delimiters or escape mechanisms, multi-token codewords, encodings that intentionally exploit tokenizer merges, arithmetic/range coding over sequences, model-internal binary channels, or provider-specific structured/file interfaces.

5 Empirical Validation

5.1 Round-trip tests

The test suite covers ordinary byte payloads, random binary data, empty input, all 256 single-byte values, multiple supported bit widths, both primary `tiktoken` encodings, larger random payloads, Direct-ID round-trips, the included SentencePiece test model when available, and CRC-32 corruption detection.

These tests establish implementation behavior for the tested configurations. They do not establish that a generative language model will copy arbitrary encoded sequences without modification.

5.2 Negative result: pairwise checks are insufficient

A useful failure mode emerged during development. Candidate tokens can pass a simple self-concatenation check such as

$$|T(w \cdot w)| = 2$$

yet fail when heterogeneous candidate strings are concatenated in longer sequences. Greedy tokenization can create context-dependent boundaries. ByteToken therefore does not treat a self-pair test as a proof of arbitrary concatenation safety for all candidate tokens.

Direct-ID mode avoids this string re-tokenization problem because it operates on token IDs directly. It introduces a different deployment constraint: the receiving interface must preserve those IDs as IDs.

5.3 Benchmark methodology

`benchmarks/benchmark_realworld.py` generates developer-like payloads: JSON records, pytest-style output, CSV rows, Python source, build-log text, an embedding vector, and random bytes. The historical filename says “realworld”, but the payloads are programmatically generated and should be described as **synthetic developer-like benchmarks**.

For each payload, the script measures Base64, ByteToken-15, LZMA+Base64, LZMA+ByteToken-15, and LZMA+DirectID-17. The target tokenizer is `o200k_base` for token counting.

The benchmark is useful for controlled comparison, but it is not a production workload study. A publication-quality follow-up should add versioned external corpora and real agent/tool traces.

5.4 Interpreting compression results

For structured text and logs, LZMA can drastically shrink the byte stream before either Base64 or ByteToken is applied. Therefore an LZMA+ByteToken end-to-end reduction should not be attributed to ByteToken alone.

The relevant comparisons are at least: (1) Base64 vs. ByteToken on the same uncompressed bytes; (2) compressed Base64 vs. the same compressed bytes encoded with ByteToken; and (3) total end-to-end size, while separating compressor and representation effects.

6 Deployment Interpretation

6.1 Transport, not model cognition

An encoded byte sequence is not a semantic representation that a language model can necessarily reason over. If an application must decode the payload before the model can use its contents, the decoded content still has to be represented to the model somehow. ByteToken is best interpreted as a transport/storage boundary optimization for systems in which encoded bytes genuinely need to cross a tokenized interface.

6.2 Artifact references may be better

Agent systems often keep large artifacts outside model context and send identifiers, summaries, or retrieved slices instead. When that architecture is available, avoiding transmission of the full payload can save more tokens than any denser encoding. A realistic evaluation should therefore compare ByteToken not only with Base64, but also with an artifact-store/retrieval baseline.

6.3 Hosted API constraints

Direct-ID results are local representation results unless a specific inference interface is documented to accept arbitrary pre-tokenized input IDs. Text APIs should not be assumed to provide such an interface. Serializing token IDs as JSON and then submitting that text causes another tokenization step and does not preserve the claimed Direct-ID token density.

6.4 Copy-through is not guaranteed

Lossless encode/decode functions do not imply lossless passage through unconstrained model generation. If an LLM is asked to repeat an encoded sequence, it may alter, omit, normalize, or truncate it. A deployment that depends on model-mediated copy-through needs explicit end-to-end fidelity experiments and integrity checks.

7 Limitations

1. **Tokenizer/version dependence.** Candidate alphabets depend on the exact tokenizer implementation and version.
2. **No universal BPE theorem.** Space-prefixed behavior is empirically useful in the tested encodings, not a theorem of BPE.
3. **Fixed-width bound is model-scoped.** Proposition 1 applies only to the defined fixed-symbol encoding model.
4. **Direct-ID deployment is limited.** It requires an interface that accepts and preserves token IDs directly.
5. **Synthetic benchmark corpus.** The included developer-payload benchmark is generated, not an independent corpus of real-world traces.
6. **Compression dominates some totals.** Large end-to-end reductions on structured data may be mostly due to LZMA.

7. **No model reasoning benefit is demonstrated.** ByteToken does not make arbitrary binary content intelligible to the model.
8. **No error correction.** CRC-32 detects corruption but does not repair it.
9. **Partial formalization.** Formal-specification work exists, but concrete tokenizer semantics are not presented as fully machine-checked.
10. **Performance is environment dependent.** Native/backend speedups require reproducible hardware/software-specific benchmarks.
11. **Security.** Dense encodings can obscure payloads from text-oriented inspection; applications must enforce authorization, size limits, and content handling before/after decoding.

BPE-dropout is one example of a tokenizer variant whose stochastic behavior differs from deterministic inference assumptions [9]; ByteToken should fail closed rather than extrapolate alphabet safety to such settings without direct validation.

8 Reproducibility

A minimal reproduction should record repository commit, Python version, `tiktoken` version, encoding name, encoder mode and bit width, payload source and SHA-256, compression settings, token-counting tokenizer, latency methodology, and round-trip hash comparison.

```
python -m pip install -e ".[all,dev]"
python -m pytest tests.py -v
python benchmarks/benchmark_realworld.py
```

The repository’s paper workflow compiles this source, uploads the PDF as a CI artifact, and publishes the validated PDF at `paper/bytetoken_paper.pdf` on the main branch. Intermediate LaTeX build files are not committed.

9 Future Work

The most useful next experiments are: evaluate on versioned external corpora; record real agent/MCP traces and identify cases where binary payloads actually cross model context; compare Base64, Base85, compressed Base64, compressed ByteToken, artifact references, and selective retrieval; report latency distributions and memory on declared hardware; test tokenizer upgrades and fail closed when an alphabet fingerprint changes; validate Direct-ID deployment only against documented inference interfaces; test model-mediated copy-through separately from deterministic transport; and strengthen the formal model where claims depend on it.

10 Conclusion

ByteToken demonstrates that tokenizer vocabulary items can be repurposed as a lossless binary transport alphabet under explicitly tested boundary conditions. The core construction is simple and reproducible: validate a symbol set, select 2^b symbols, and map fixed-width bit chunks to those symbols.

The interesting engineering question is where the representation is actually useful. String-mode gains depend on tokenizer behavior; Direct-ID gains depend on the inference interface; structured-data totals depend heavily on conventional compression; and many agent systems can avoid transporting the payload entirely by using artifact references and retrieval.

ByteToken should therefore presently be read as an experimental tokenizer-aware transport primitive with a scoped theoretical result and an open reproducibility surface, not as a universal context-compression protocol.

References

- [1] Tao Ge, Jing Hu, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. *arXiv preprint arXiv:2307.06945*, 2023.
- [2] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376. Association for Computational Linguistics, 2023.
- [3] Simon Josefsson. The base16, base32, and base64 data encodings. Technical Report RFC 4648, Internet Engineering Task Force, 2006.
- [4] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71. Association for Computational Linguistics, 2018.
- [5] David Nicol. base32768: Binary-to-text encoding highly optimised for utf-16. <https://github.com/qntm/base32768>. Accessed 2026-08-30.
- [6] OpenAI. tiktoken: a fast BPE tokenizer for use with OpenAI’s models. <https://github.com/openai/tiktoken>, 2023. Accessed 2026-08-30.
- [7] Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E. Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srini Iyer. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258. Association for Computational Linguistics, 2025.
- [8] Igor Pavlov. LZMA sdk. <https://www.7-zip.org/sdk.html>. Accessed 2026-08-30.
- [9] Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1882–1892. Association for Computational Linguistics, 2020.
- [10] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725. Association for Computational Linguistics, 2016.
- [11] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.